Godot 可破坏建筑与整楼坍塌技术深研

Executive Summary

你记忆里的 NVIDIA “可破坏建筑”技术，大概率是较老的 `APEX Destruction`；在 NVIDIA 当前公开的破坏链路里，更应该看的其实是它的后继 `Blast`。[1][2][3] 但关键现实是：`Blast` 自己并不提供物理体、碰撞体或渲染表示，它只是一个“分块/破坏求解 SDK”，真正落到 Godot 里，仍然需要你自己做一整层 GDExtension、场景节点、碰撞、渲染、流式状态同步的桥接。[1][4][5]

如果目标是“攻击游戏里的任意一座大楼，都能最终坍塌”，那么对你当前这个 `70km x 70km` 流式城市原型来说，最现实的路线不是“给每一栋楼做全实时高精度碎裂”，而是做一个混合方案：所有楼都支持统一的结构伤害状态机；近距离少量楼支持英雄级预分块坍塌；坍塌结束后再退化为静态瓦砾代理体。这条路线最符合 Godot 当前生态、你项目的 streaming 架构，以及可控的性能预算。[1][6][7][8][9]

Key Findings

- **NVIDIA 这条线目前应看 `Blast`，不是旧 `APEX`**：`Blast` 官方明确写明它 intended to replace APEX Destruction；但它当前仍是“刚体、预分块（pre-fractured）破坏”模型，不是你给任意完整建筑打一炮就即时体素化碎裂的黑科技。[1][2]
- **Godot 能接 NVIDIA 技术，但不会开箱即用**：Godot 官方文档说明，GDExtension 可以在运行时加载 native shared libraries，并在 `.gdextension` 里声明第三方依赖库；这意味着你理论上可以包 Blast/PhysX 进去，但整个桥接层需要你自己做。[4][5]
- **Godot 官方不会把 GameWorks 这类闭源 SDK 直接做进核心**：Godot FAQ 明确说，核心团队没有计划支持这类第三方闭源/专有 SDK，但你可以自己做模块或插件来接。[10]
- **Godot 社区确实已经有“类似思路”的方案**：当前可见的社区路线至少有三类：预分块刚体路线（`Destructibles CSharp`、`Destruction`）、体素破坏路线（`Voxel Destruction`）、以及更底层的体素世界/可编辑体积地形路线（`Voxel Tools`）。[6][7][8][9]
- **“任意一座大楼都真实碎一地”在全城尺度上不是技术不可做，而是预算不可做**：Godot 官方碰撞文档明确建议动态对象优先 primitive/convex 形状，而 concave/trimesh 只适合 `StaticBody`，且大量 collision shapes 会让窄相位成本明显上升。[8] 这直接限制了“每栋楼数百动态碎块 + 每块复杂碰撞”的全城常开方案。

Detailed Analysis

1. NVIDIA 那条“建筑可破坏”技术，到底是哪条线

如果按“爆炸后建筑碎一地、还能做支撑结构、能做整栋坍塌”这个记忆去反推，你想起的大概率是 `APEX Destruction + PhysXLab` 那条老线。[3] NVIDIA 在 APEX 文档里明确把 destructible asset / destructible actor / destructible actor joint 当作核心对象，并且支持“support chunks”“inter-actor support”，官方甚至直接举了“用更小的 destructible building blocks 组装成更大的 destructible structure”的例子。[3]

但从 NVIDIA 自己后续公开资料看，正式接班的是 `Blast`。Blast 官方 README 和 API 文档都明确写了两件关键事实：

1. `Blast` 是 intended to replace APEX Destruction。[1][2]
2. `Blast` 当前设计目标是 **rigid body, pre-fractured destruction**，也就是“预先分好块的刚体破坏”。[2]

这句话非常重要。它意味着：

- 它不是“任意完整建筑，运行时随便打一炮，就自动高质量实时碎裂”。
- 它更像是一个高性能的“分块关系 + 伤害传播 + actor split”求解器。
- 你要先有 chunk hierarchy / support graph / bond 这些前置数据。[1][2]

换句话说，NVIDIA 这条线更适合“先做/导入可破坏资产，再在运行时驱动其破坏”，而不是“把整个程序生成城市在运行时临时变成 fully destructible voxel world”。

2. Godot 里能不能直接利用 NVIDIA Blast / APEX

能接，但不是直接能用。 这件事在 Godot 官方文档里其实很清楚。

Godot 官方对 GDExtension 的定义是：它允许引擎在运行时与 native shared libraries 交互，并运行不需要重新编译引擎的原生代码。[4] `.gdextension` 文件又允许你在导出时声明并打包依赖的动态库，例如 Windows 下的多个 DLL。[5]

这意味着从“技术接入”角度讲：

- 你完全可以把 `Blast` 当作一个第三方 C++ SDK 接进 Godot。
- 你可以用 `godot-cpp + GDExtension` 包一层 `BlastBridge`。
- 你可以在 Godot 场景里暴露诸如 `BlastAssetResource`、`BlastActorNode3D`、`apply_damage(world_pos, radius, energy)` 这样的接

口。

但问题在于，`Blast` 官方 README 明确写明：

- `NvBlast / NvBlastTk` **没有 physics or collision representation**。[1]
- 也 **没有 graphics representation**。[1]
- 它是 physics/graphics agnostic，需要用户自己创建这些表示。[1]

这对 Godot 项目意味着你至少还要自己解决：

- `Blast actor/chunk` 到 `Node3D / MeshInstance3D / RigidBody3D` 的映射。
- chunk 碰撞形状生成与切换。
- Godot 物理世界和 Blast 破坏事件的同步。
- streaming chunk 卸载/重载后的破坏状态持久化。
- 远处 LOD、近处高精度破坏、坍塌后的瓦砾代理体切换。

更麻烦的是，Blast 虽然带 `ExtPhysX` 参考实现，但那是“PhysX actor/joint manager”，不是 Godot 原生桥。[1] 你要么把破坏对象的一整套物理也交给 Blast/PhysX，再和 Godot 世界做双向同步；要么只用 Blast 做 fracture/state，物理仍然交给 Godot 的 `RigidBody3D`。前者工程复杂，后者桥接工作更多。

我的判断是：**Blast 在 Godot 里是可接入的，但它更像一条“引擎级专项研发线”，不是短平快功能线。**

3. Godot 社区目前有哪些“类似技术”

Godot 社区已经出现了几条很像的方案，但它们代表的是不同思路。

`Destructibles CSharp` 这条路线是最接近传统“预分块刚体破坏”的。Godot Asset Library 上的说明写得很直白：它把 shard mesh 转成带自定义碰撞的 mesh instances / rigid bodies，每个 shard 有自己的 rigid body；这些 shard 既可以在游戏里生成，也可以预先生成，并且设计上就是配合 Blender 的 cell fracture 一类工具使用。[6] 这和 Blast 的“预分块资产”思路非常接近，只是它是 Godot 生态内的实现。

`Destruction`（Jummit）则更轻量，它的资产库描述是“把对象替换成 shattered version”或者“把 mesh 列表转成 RigidBodies”。[7] 这类插件更像是玩法层的破坏替换器，适合做“物体被击中后替换成碎块版本”，但不天然解决大楼级结构支撑、长时流式状态、远近 LOD 等大问题。

`Voxel Destruction` 和 `Voxel Tools` 则属于另一类世界观。`Voxel Destruction` 当前资产库描述为：支持动态 voxel destruction、debris、`.vox` 导入和 specialized destruction node。[9] `Voxel Tools` 则更底层，它是一个 C++ module/extension，提供可在运行时编辑的 3D 体积地形、Godot physics integration，以及 paging chunks in and out 的无限体积地形框架。[11]

这两条线的优势是“破坏自由度高”，缺点是：

- 你的建筑内容管线会从 mesh/building scene 变成 voxel content pipeline。
- 体素存储、重建网格、碰撞刷新、流式保存都会变成核心系统问题。[11]

所以，社区并不是没有技术，而是“每一种都在不同维度上收费”。

4. 想做“任意一座大楼都能塌”，Godot 里的真实约束是什么

这里真正的硬约束，不是“能不能做坍塌动画”，而是“能不能在你的城市规模下，长期稳定地让任何楼都进入可破坏状态”。

Godot 官方碰撞文档给了几个非常重要的性能边界：

- 对 `RigidBody3D` / `CharacterBody3D` 这类动态对象，官方推荐优先 primitive shapes，因为行为最可靠、性能通常也更好。[8]
- convex shape 可以用于较复杂动态物体，但 shape 数量一多，这个优势会消失。[8]
- concave/trimesh 是最慢的，而且只能安全用于 `StaticBody`，不能作为常规动态碎块碰撞方案。[8]
- `StaticBody` 如果挂了很多 collision shapes，广相位优化会失败，窄相位会被迫逐个检查 shape。[8]

把这些限制翻译成“整楼坍塌”语境，大概就是：

- 你不能让每栋楼都预备 500 个动态 concave 碎块。
- 你也不能把所有大楼都永久保持“随时可切换成几百刚体”的高开销状态。
- 你必须把“远处可见性”和“近处高精度物理”拆开。

另外，`SoftBody3D` 不是这类问题的正解。Godot 文档把它定义为 deformable objects，例如 cloth。[12] 它更适合布、软组织、弹性体，而不是钢筋混凝土或框架结构楼房整体破坏。

5. 对你这个项目，我推荐什么路线

结合你当前仓库的现状，这是我的明确建议：

推荐路线：做“全城统一结构伤害 + 近处英雄级坍塌”的混合方案，而不是追求全城 fully physical destruction。

可以分三层：

A. 全城通用层：结构伤害状态机

所有建筑都具备以下通用状态：

- `intact`

- `damaged_local`
- `critical`
- `collapsing`
- `collapsed`

攻击只要命中建筑，就可以累计 damage、记录命中层位和结构点。到达阈值后，不是立即生成海量碎块，而是切入坍塌序列。

这层的好处是：**你可以说“任意一栋楼都可被打塌”，但并不要求任意一栋楼都始终处于高成本物理求解状态。**

B. 中近景层：预分块英雄坍塌

只对玩家附近、任务相关、或真正被打进 `critical` 的建筑，切入预分块坍塌资产：

- 少量结构块：20 到 80 个大块，而不是几百上千块。
- 主要承重块用 `RigidBody3D + primitive/convex`。
- 小碎屑、玻璃、粉尘、墙皮用 GPU 粒子、Decal、instanced rubble 伪装。
- 坍塌结束后，把整堆结果烘成单个或少量静态瓦砾代理体。

这条线很像 Blast/APEX 的“support + chunk actor split”玩法，但更贴近 Godot 的性能边界。[1][3][8]

C. 限量实验层：体素化地标/试验区

如果你非常想做“炮打哪碎哪、洞可以挖穿、楼体能被一点点啃掉”的感觉，那就别上全城，而是挑：

- 1 个试验街区
- 1 座特殊地标
- 或者 1 类低层建筑

用 `Voxel Destruction` / `Voxel Tools` 做 PoC。[9][11]

这样你既能验证“高自由度破坏”玩法值不值，又不会把全城内容管线一次性掀翻。

6. 是否值得直接上 NVIDIA Blast

我的结论是：

- **作为研究项目，值得。**
- **作为你当前主线的第一落地点，不值得。**

原因不是 Blast 不强，而是它离“Godot 场景里真的塌一栋楼”还隔着太多 Godot 专属桥接工作。[1][4][5]

如果你坚持走 Blast，我建议把它当成单独专项：

1. 先做一个 `Blast via GDExtension` 的技术 spike。
2. 只打通一个单楼 demo。
3. 明确 damage -> split -> chunk rigidbody spawn -> settle -> rubble proxy 的完整链路。
4. 跑近景性能和保存/重载验证。

只有这一条真的稳了，才考虑它是否值得进入城市主线。

Areas of Consensus

- NVIDIA 旧破坏线是 `APEX Destruction`，新线是 `Blast`。[1][2][3]
- `Blast` 的核心模型是 **预分块刚体破坏**，不是天然运行时全自由碎裂。[2]
- Godot 可以通过 GDExtension 接第三方原生库，但官方不会把闭源 GameWorks 直接纳入核心支持。[4][5][10]
- Godot 社区已经有预分块和体素两条可破坏路线，只是成熟度和适用场景不同。[6][7][9][11]
- 对大世界项目来说，真正难点是性能、流式、保存状态、碰撞预算，而不是“单次演示里能不能炸塌一个模型”。[1][8][11]

Areas of Debate

- **是否要用 NVIDIA Blast**：从“技术含金量”和“结构性破坏表达力”看，Blast 很强；但从“Godot 项目接入成本”看，社区内实现或自研预分块方案通常更划算。[1][4][5][10]
- **是否要 voxelize 建筑**：体素路线自由度最高，但会重塑你的资产与 streaming 架构；对单地标非常诱人，对全城很危险。[9][11]
- **“任意一栋都能塌”到底要多真实**：如果用户要求的是“玩法上最终可坍塌”，那状态机 + 预分块代理就够；如果要求“每个结构点都真实传力并逐块倒塌”，那研发成本会跃迁一个数量级。[1][3][8]

Decision Matrix

路线	技术自由度	Godot 接入成本	全城扩展性	近期推荐度
`Blast + GDExtension`	高	很高	中	中
预分块刚体（Godot 原生/社区插件）	中高	中	高	高

路线	技术自由度	Godot 接入成本	全城扩展性	近期推荐度
体素破坏（Voxel Destruction / Voxel Tools）	很高	高	低到中	中
纯演出假塌（动画/替换）	低	低	很高	中

Recommended Next Step

如果你要我给一个非常实际的 next step，我建议这样排：

1. **先做 v33 技术预研，不直接全城铺开。**
2. **只挑一栋中高层建筑做“可被打塌”的完整垂切样机。**
3. **实现三段式表现：**
   - 远景：普通建筑
   - 近景受损：局部破坏 + 局部碎块
   - 临界坍塌：预分块大块体 + dust/debris + 最终 rubble proxy
4. **把 streaming/save-state 一起验证。**
5. **如果样机成立，再决定要不要专门起一个 Blast/GDExtension 分支。**

对你当前项目，我的推荐次序是：

- 第一优先：`Godot 原生混合坍塌方案`
- 第二优先：`局部体素试验区`
- 第三优先：`Blast/GDExtension 专项研发`

Sources

[1] NVIDIA GameWorks, “Blast 1.1.10 README,” GitHub. 官方仓库，一手资料。
https://github.com/NVIDIAGameWorks/Blast

[2] NVIDIA, “NVIDIA Blast SDK 1.1 API Reference: Introduction.” 官方文档，一手资料。
https://docs.nvidia.com/gameworks/content/gameworkslibrary/blast/1.1/api_docs/files/pageintroduction.html

[3] NVIDIA, “Destruction Introduction — NVIDIA APEX Documentation.” 官方文档，一手资料，但属于旧技术线。

https://docs.nvidia.com/gameworks/content/gameworkslibrary/physx/apexsdk/APEX_Destruction/Destruction_Module.html

[4] Godot Engine Docs, “What is GDExtension?” 官方文档，一手资料。
https://docs.godotengine.org/en/stable/tutorials/scripting/gdextension/what_is_gdextension.html

[5] Godot Engine Docs, “The .gdextension file.” 官方文档，一手资料。
https://docs.godotengine.org/en/stable/tutorials/scripting/gdextension/gdextension_file.html

[6] Godot Asset Library, “Destructibles CSharp.” 官方资产库页面，社区插件一手描述。
https://godotengine.org/asset-library/asset/1850

[7] Godot Asset Library, “Destruction.” 官方资产库页面，社区插件一手描述。
https://godotengine.org/asset-library/asset/2189

[8] Godot Engine Docs, “Collision shapes (3D).” 官方文档，一手资料。
https://docs.godotengine.org/en/stable/tutorials/physics/collision_shapes_3d.html

[9] Godot Asset Library, “Voxel Destruction.” 官方资产库页面，社区插件一手描述。
https://godotengine.org/asset-library/asset/3743

[10] Godot Engine Docs, “Frequently asked questions: Will [insert closed SDK such as FMOD, GameWorks, etc.] be supported in Godot?” 官方 FAQ，一手资料。
https://docs.godotengine.org/en/4.4/about/faq.html

[11] Zylann, “Voxel Tools for Godot,” GitHub README. 项目官方仓库，一手资料。
https://github.com/Zylann/godot_voxel

[12] Godot Engine Docs, “Using SoftBody3D.” 官方文档，一手资料。
https://docs.godotengine.org/en/stable/tutorials/physics/soft_body.html

Gaps and Further Research

- 我没有在本次调研里深入评估 `Blast` 在 2026 年是否存在更私有或企业内的后续版本；公开可验证资料里，GitHub 仓库显示的最新 release 是 **2020-01-13**。[1]
- 我没有对 Godot 社区这些 destruction 插件做源码级 benchmark，本次结论主要基于官方资产页/仓库说明，而不是实测 FPS 数据。[6][7][9][11]
- 对你这个仓库真正重要的下一步，不再是“继续搜资料”，而是做一个 **单楼坍塌样机**，把节点数、刚体数、碰撞形状数、chunk unload/reload 状态和瓦砾代理体切换都跑出来。